Quiz 01 - Answers

1. Describe the difference between a Docker Image and a Docker Container.

- A **Docker Image** is a lightweight, standalone, and executable package that contains everything needed to run a piece of software, including the application code, runtime, system tools, libraries, and dependencies.
- A **Docker Container** is a running instance of a Docker Image. When an image is executed, it creates a container, which is an isolated environment where the application runs. Containers are ephemeral, meaning they can be started, stopped, and removed without affecting the underlying image.

Key Difference:

- A **Docker Image** is like a blueprint or template.
- A **Docker Container** is the live, running version of that image.

---

2. Explain the concept of resource multiplexing in an Operating System.

- **Resource multiplexing** is the technique by which an operating system (OS) manages and shares limited system resources (CPU, memory, I/O devices) among multiple processes efficiently.
- The OS ensures that different applications and users can access system resources without interference using various scheduling and allocation strategies.

Examples of Resource Multiplexing:

- **CPU Scheduling:** The OS assigns CPU time to multiple processes using scheduling algorithms (e.g., Round Robin, Shortest Job First).
- **Memory Management:** The OS allocates RAM to different processes dynamically using paging or segmentation.
- **I/O Multiplexing:** Allows multiple processes to share I/O devices, such as multiple applications using the same network interface.

Importance:

- Ensures efficient use of resources.
- Prevents resource starvation and deadlocks.
- Improves system performance and responsiveness.

---

3. How does a Docker container differ from a traditional Virtual Machine in terms of architecture?

Docker Containers and **Virtual Machines (VMs)** both provide isolated environments, but they differ in how they achieve this isolation.

Key Differences:

Feature	Docker Container	Virtual Machine (VM)
Architecture	Shares host OS kernel	Runs a full OS on a hypervisor
Startup Time	Starts in seconds	Takes minutes (OS boot required)
Resource Usage	Lightweight, uses fewer resources	Heavy, consumes more RAM & CPU
Performance	Faster, less overhead	Slower due to full OS virtualization
Isolation	Process-level isolation	Full OS-level isolation

Why is Docker more efficient?

- Containers **share** the host OS kernel, so they do not need to boot an entire OS, reducing overhead.
- VMs run on a **hypervisor**, which requires each VM to have its own OS, making them more resource-intensive.

---

4. Explain system call briefly.

- A **system call** is a way for a user-space application to request a service from the operating system's kernel.
- Since user applications do not have direct access to hardware, they use system calls to perform operations such as file management, process control, and network communication.

Examples of System Calls:

- **Process Management:** `fork()`, `exec()`, `exit()`
- **File Management:** `open()`, `read()`, `write()`, `close()`
- **Memory Management:** `mmap()`, `brk()`
- **Networking:** `socket()`, `connect()`, `send()`, `recv()`

Example in C:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    write(1, "Hello, System Call!\n", 20);
    return 0;
}
```

---

5. How does a Docker container start faster than a Virtual Machine?

- Docker containers start faster than virtual machines because they **do not require a full OS boot**.
- Containers run as **isolated processes** on the host OS, sharing its kernel. This eliminates the overhead of booting a separate OS for each container.

Reasons for Faster Startup:

- **No OS Boot:** Containers do not have a separate guest OS, unlike VMs.
- **Lightweight Architecture:** Only the application and dependencies run in an isolated environment.
- **Uses Host Kernel:** No need to emulate hardware or load a new OS instance.

Example Startup Time:

- **Docker Container:** Starts in milliseconds to a few seconds.
- **Virtual Machine:** Can take 30 seconds to a few minutes, depending on the OS.

---

6. Explain the difference between multiprocessing and multicore systems.

- **Multiprocessing** and **multicore** systems both involve executing multiple tasks simultaneously, but they differ in how they achieve parallelism.

Key Differences:

Feature	Multiprocessing	Multicore
Definition	Uses multiple processors (CPUs) in a system to run processes in parallel.	A single CPU contains multiple cores that execute tasks independently.
Execution	Each processor has its own control unit and can run separate processes.	Each core in a processor can execute its own thread or process.
Example	A server with two physical CPUs, each handling separate tasks.	A laptop with a quad-core processor (4 cores in one chip).
Efficiency	Increases performance but requires multiple CPUs.	More power-efficient, as cores share the same chip.
Common Usage	High-performance computing, servers, and mainframes.	Modern laptops, desktops, and mobile processors.

Can both exist on the same PC?

- **Yes!** A system can have multiple **physical CPUs (multiprocessing)**, and each CPU can have multiple **cores (multicore)**.
- Example: A high-end workstation with **2 processors (multiprocessing), each having 8 cores (multicore), totaling 16 cores**.